



The Lazy Developer

02

MODULE

Setting Up Your AI Coding Tool (Cursor, Claude Code & more)

Let's go over the key AI coding tools and understand their features. Then we can dive into tips that will make your AI-assisted development experience more enjoyable.

Table of Contents

- 01. Quick overview of AI coding tools (8 min)
- 02. Setting Up Your Development (Dev) Environment (10 min)
- 03. Understanding AI coding rules and configuration (8 min)
- 04. Key files you need in any project (11 min)
- 05. Chat optimisation hacks (10 min)
- 06. Build out your Products Requirement Doc (PRD) (13 min)
- 07. Module 2 recap: let's play a game! (30 mins)

"Imagine Lego got bitten by a radioactive AI..." — that's pretty much any LLM for coding in a nutshell. It's everything you need to build something amazing, but comes with AI. You orchestrate the construction but everytime you get stuck, you can just say, "fix this." Cursor keeps the familiar look-and-feel of a desktop code editor, but tucks an LLM right next to your files so you can ask, edit and run without ever leaving the window. Let's walk through the bits you'll use on day one.

Opening the hood

When you launch Cursor for the first time you'll land on something that looks suspiciously like VS Code because it is VS Code under the hood, just with extra AI plumbing. If you don't know what VS Code is, that doesn't matter either frankly! It's just a code editor that's very popular and what has helped Cursor become so popular among developers as it's like working in the same environment they're already used to.

Here is where you choose what LLM you want to use. Cursor now tells you if advanced models will cost you per question you ask it. Hint: Stick to the free ones unless you've got money to burn!

The application is split into 3 distinct sections:

1. File Explorer (far left). This is the IKEA shelving of your project: folders, files and a little '+' button to add new ones.
2. Editor Pane (centre stage). The big canvas where your code (or soon-to-be code) lives. Standard line numbers on the left, tabs across the top.
3. Chat/Diff Pane (right-hand column). Here's the secret sauce: a rolling conversation with the AI plus an inline diff viewer. Ask a question, preview the suggested change, then thumbs-up or thumbs-down it.
4. A Mini Console Window (below). Here you can run your own little terminal window (or multiple ones), look at active problems detected in files, run your client (front end) and server (back end) locally for development, and see the logs that appear if you code them in.
5. Status Bar (very bottom). A neat little badge shows which model is active (e.g., GPT-5.2). Click it to swap brains later on.

What about Claude Code, or others?

If you're running these within VS Code or Cursor, then they can run from a terminal window. Some people (like me) prefer that and it's still connected to Cursor or VS Code. However, now, these platforms are almost all accessible via web browser, or as a stand alone app. I would always suggest a stand alone app model as you're in more control of the environment and the data. You're not always relying on an internet connection just to power the UI. At least you can browse your code locally and get to files. Web-based code builders limit that and you need to log in before you can see anything.

Anyway, if you don't know what do to, talk to your AI and tell it about which environment works best based on your technical ability. Then it will help you set that up so you can get started.

Mini-glossary (jargon you'll bump into)

Term	Human-speak equivalent
Repo	Short for "repository" and is used to label your project folder. It's also used in Github as the project you're working on.
Diff	A before-and-after picture of the file. Green = added, Red = removed. (see image below)
Accept	"Yep, ship it." Merges the AI's patch into your file.
Discard	"Nope." Tosses the suggestion away.
Context window	The AI's short-term memory. Too much text? It forgets older lines.
Model	The brain doing the work (Gemini, Claude, etc.).

Understanding these six words is 80 % of the technical vocabulary you'll need for the whole course.

Here's a Diff example. Accept or Reject it?

Best-practice quick hits

- Start Small: On day one, point your AI at a tiny project (even a single .py or .ts file). It's like learning to drive in an empty parking lot.
- Talk in Goals, Not Steps: Instead of "type print," say "Show a greeting on-screen." The AI fills the gaps.
- One Change at a Time: Accept or discard after every preview. Long chains of unaccepted orange boxes can get messy fast.
- Narrate Your Rejections: If you scrap a suggestion, tell the model why ("Too verbose" or "Wrong function name"). Future answers improve.
- Screenshot Troubles: When something looks weird, grab a screenshot and paste it into chat. AI can read images and debug UI glitches.

That's the whirlwind tour! Treat AI like a friendly pair programmer: point to some code, ask a plain-English question, preview the fix, and accept when you're happy. In the next section we'll slow down and build your local "sandbox" so you can experiment without fear of wrecking a production server. So, go and ahead and start talking! What are you going to build today?

Goal: install the must-have tools, spin up a safe “localhost” sandbox, then let AI write five different starter apps for you with nothing but natural-language prompts.

Install your toolbox

On your learning dashboard, expand the Resrouces section and you will see "installation guide" which will help you to setup your dev environment. Of course, you can be lazy and just ask AI to help you with that. However that overview gives you a starting point to get the foundation in place!

Tool	Why you need it
Homebrew / Winget / Chocolatey	One-line package managers — install everything with brew install or winget install.
Node.js + npm	A JavaScript runtime and the package installer
Git	Cursor will call when it needs libraries. Version control; lets you undo mistakes and push code to GitHub for testing or production.
tree (optional)	Prints the folder structure so you can sanity-check what Cursor created.

As I'm a Mac user, I'll be focusing on using the tools for Mac but the same tools exist on Windows too, they just may have a different name. The good thing is, if you don't know, AI is always there to help you. It will also run the commands for you so you don't have to! It's best to do it yourself though to learn the process!. The next time you need to build an app, you'll just be that much faster.

I use the terminal inside of Cursor to do commands as I'm normally running Claude Code. It's Claude who will execute the commands. This is better as it means it's all in one window, and Claude can see the results and see what to fix based on that. You don't need Cursor at all to use Claude Code, but it integrates well into my dashboard of projects and files and allows me to see what's going on easily. I can copy files from other projects to share with Claude and say "Hey, I want this feature we did in this other project, you can see the files here..." so Claude can go look.

Why a dev !' test !' prod pipeline matters

If you're going to do this properly when you build your first real app, then you need to have some way to build and test without breaking a working version of your app. This is even more important if you have deployed an app and it has users on it. The last thing you need to be doing is live changes to things that are in production without testing them first.

- 1. Dev = your computer. Messy, experimental, private.
- 2. Test / staging = a shared server where you click around and break things safely. This will be hosted somewhere which is more like a clone of your production environment so should be very similar. It's like pre-production. Websites like Vercel and Railway will give you a hobby account for free so you can set this up. (Links to these platforms are in the Resources section.
- 3. Prod = the live site that customers touch, where you may generate revenue, grab customer data. This one you don't mess with.

We will discuss in a later module all you need to know about Git and Github but for now it's just good to have Git installed and go sign up to Github.com for free.

Meet localhost

When you have your client installed (eg. Vite), you can then open Chrome or Safari and browse to `http://localhost`, you're talking to your own machine. It's lightning-fast and free, but remember it's not 1-to-1 with production. Notice there's no "s" in "http://" as you don't have a security certificate. Everything is running on one machine, inside the same network. This is not realistic of production at all but it's great to build something locally and test it to make sure the vast majority of the functionality works. You can still connect it to other sites for things like authentication, databases, and calling other APIs.

Things that can differ to production:

- Permissions – you have admin rights locally; your cloud host probably won't.
- Networking – company VPNs, firewalls, or HTTPS certificates add layers you don't see at home.
- Security – browsers may block cross-site cookies when online or only in "real" HTTPS environments.

Treat localhost like a rehearsal studio: perfection isn't required, but practice saves on-stage embarrassment.

If you want to run more than one app locally, can you do that too! You would simply change the port number (or have the AI do it for you). This means the default port when you go to `http://localhost` is port 80, but you can have your webserver for each app have a different port number. Like 1000, 1001, 1002... so when you go to `http://localhost:1000` you will see something different to the default port (80). You will just be creating a new web server each time in different folders and can run them independantly.

For axample, if you have AI build you a Vite web server, it has a configuration file where it will set the port for you and tell you how to access it. If your dev machine has wifi and a connected ethernet cable, you can even run Vite in a way that allows others in your home network to see the site from another laptop or phone. Great for having another user in the family test. (hint: you would start the web server with the command `npm run dev -- --host`).

When you'll need Docker Desktop

For these front-end demos Node's simple server is plenty. The minute you add:

- A database (Postgres, Mongo)
- A back-end API (Flask, Express)
- OS-specific dependencies (ImageMagick, ffmpeg)

...Docker becomes the sandbox that guarantees “works on my machine” also works on the test server. Docker is a "container" that runs everything it needs inside of it. Calls are made to the container to grab what it needs and it runs independantly. It's not one app, it's a bunch of apps and runtime libraries all contained. Containers are widely used in production architecture for their ease and flexibility. AI can even write your Dockerfile and help you with everything you need to set it up, just ask! (eg. "Help me to setup Docker Desktop on my computer").

Best-practice cheat-sheet

- One folder per mini-app. Keeps dependencies isolated and AI's suggestions laser-focused.
- Prompt like a product manager: include the goal, tech limits, and folder path.
- Accept early, commit often. Small diffs are easier to understand and roll back.
- Start saving useful info in the README. For example `npm run dev -- --host` will allow Vite to be accessible by the local IP address in your home network.

Rules: Your “House Style” File

AI is the chef; and a rules file is the recipe card you slip under the kitchen door.

When AI knows your tastes (tech stack, naming quirks, lint rules), its suggestions stop feeling “generic GitHub” and start sounding like your codebase.

Start with a chat about your stack

Before you write a single rule, spend five minutes talking to AI (Cursor or ChatGPT / Claude) about the tech choices for this project. That clarifies what the rules should lock in.

Sample discovery prompts

We’re building a SaaS dashboard. But I really have no idea where to begin. I need something...

I’m a just starting out and ready to build my first web app. It’s going to be a motivator ...

Use the answers to decide:

- Languages (TypeScript vs JS, Python vs Go)
- Framework flavours (Next.js vs Vite, Flask vs FastAPI)
- Testing / linting tools (Playwright, ESLint, Ruff)

Everything you nail down here becomes a rule that keeps AI from drifting later.

Where rules live

Cursor processes rules from two places every time it generates code:

1. Global settings – open Settings !’ Rules in the top right gear icon. Rules here apply to every project you open.
(insert screenshot of Cursor settings panel highlighting the Rules tab)
2. Project file – a plaintext file named `.cursorrules` sitting at the root of the repo. These override or add to the global ones only when that folder is active.

Why both? Global rules handle personal habits (“Always write comments like I’m five”). Project level rules handle stack specific demands (“Use Tailwind utility classes, avoid inline styles”).

Claude Code has its own markdown file (it’s called `claude.md`), and so does Codex. Sometimes for the web-based apps these are found in the system settings. If you don’t know, just ask the tool you’re using about how to setup the rules file properly! Then, ask it to write the rules for you. Every few

weeks, do a sanity check. Maybe your project has changed or added a new techie bit, so the rules should be updated to reflect that.

Best practice cheat sheet

- Be prescriptive, not verbose. Ten crisp rules beat fifty vague paragraphs.
- Lead with do, end with don't. Positive commands first, prohibitions second.
- Version your rules. Add a header: # v1.2 – 2025 05 01 – switched to Go 1.22.
- Keep secrets out. Rules live in the repo; don't paste API keys.
- Iterate. If AI drifts, update the rule instead of nagging in every prompt.
- Stay in sync with tooling. When Prettier or Black upgrades, update the rule so AI matches the formatter.
- Reference docs, not blogs. e.g. "Follow React docs for useEffect cleanup." Links age; official docs update.
- Reuse. Save proven rule sets in a personal "rules vault" and copy them into new projects.

Keeping /Your /Project /Well Documented: README, CHANGELOG, BUGS & ENHANCEMENTS in plain English

Think of these three Markdown files as your project's memory palace. They tell you, /future you, and AI what the code does, why it changed, what's not working right, and what you'd like to build next. The clearer that story, the smarter (and less repetitive) the AI's answers become.

Key thing to remember here is that you will not be writing these documents, the AI will!

You just need to remember to tell it to update these files and also tell it what you want in them!

So what's the point? Simple, it's all about context. Every time you start a new chat, load up these files into the chat window. Just highlight all the text and press "Add to chat" or copy, go to the chat window and paste. It will attach the file without all the text. Then, when you start your new chat session, the AI is instantly up to speed on what your app does and is ready to dig in to look at code, find things, and understand the big picture so it doesn't ask you silly questions!

You will be creating hundreds of chat sessions for a complex app because the context window isn't big enough to hold all the information you discuss without purging some of the data.

README.MD

A good README is helpful to the AI with this information.

1. What does this project do? - overview, website, problem statement, how it works.
2. What is the tech stack - useful to have the tech stack and configuration.
3. Key features - a list of the main features of the app
4. Share a tree view of the files and descriptions - add a one-line description to each file so you know what they do
5. How would you install this from scratch? - installation instructions, dependencies, external 3rd party integration parameters, environment variables
6. How would you deploy the project - where is it deployed and what was needed to do that?

/Suggested skeleton

```
# Mini Calculator SPA
```

A tiny web app that adds, subtracts, multiplies and divides two numbers.
Built with vanilla HTML, CSS, and JS as a demo for non coders.

```
## Quick start
```

1. Clone repo !' `cd projects/app-1`
2. Install dependencies (if any) !' `npm install`
3. Start dev server !' `npx serve . --port 2000`
4. Visit <http://localhost:2000>

```
## Folder structure
```

```
tree -L 1
```

```
# or paste the output here manually so it's visible on GitHub
```

```
.  
% % % index.html      # UI markup  
% % % style.css       # Page styling  
% % % app.js          # Calculator logic  
% % % README.md       # You're reading it!
```

5.2 / CHANGELOG /.md - "What changed, and when?"

A CHANGELOG is a curated, chronological history of notable changes /.

Here's an example:

```
## Changelog
```

All notable changes to this project will be documented in this file.

The format is based on Keep /a /Changelog and adheres to Semantic Versioning.

```
### [Unreleased]
```

```
### Added
```

- Dark mode toggle behind a `?theme=dark` URL param

```
### [0.2.0] - 2025 05 02
```

```
### Added
```

- Multiply and divide buttons

```
### Fixed
```

- NaN error on divide by zero

```
### [0.1.0] - 2025 05 01
```

```
### Added
```

- Initial add / subtract calculator

Grouping by Added / Changed / Fixed / Removed lets humans skim.

AI, seeing those headings, will reference the right version when you ask, “Show me code before dark mode was added.”

/5.3 / ENHANCEMENTS /.md - “Wouldn’t it be cool if…?”

Treat this like a wish list or product backlog:

Enhancements

Last updated: 2025 05 02

Priority	Idea	Notes
+P+P+P	Unit tests with Jest	Start with add()/subtract()
+P+P	Responsive layout	Looks cramped on phones
+P	Export history to CSV	Low demand, nice to have

Cursor reads this file too. A future prompt such as “Implement the top priority enhancement” will result in code for unit tests, not CSV export.

/5.4 / BUGS.md - “Houston, we have a problem…”

This are all the snags, issues, niggles, problems, whatever you want to call them. List them by order of severity or how urgent it is they need to be fixed.

Bugs to fix

Last updated: 2025 05 02

Severity	Issue	Notes
+P+P+P	Authentication failure	Means that users can't access their settings
+P+P	JSON error when backing up	Probably an issue in how the parameter is passed
+P	Dark mode not fully accurate	Likely it just annoys me but I need to fix this

AI reads this file too. A future prompt such as “build a plan to fix the top priority bugs” will begin the discussion around tackling the major issues in the project.

5.5 / When to update the diary

Moment	What to do
New feature accepted	Add an entry under ## [Unreleased] in CHANGELOG. Tick off the idea in ENHANCEMENTS.
Bug fix merged	Insert a Fixed bullet and link the GitHub issue (when you have that setup).

Version bump (e.g., 0.2.0 /! /0.3.0)	Copy the [Unreleased] section, paste below with today's date, then clear Unreleased for the next cycle.
Big refactor	Update README's Folder structure if files moved/renamed.
Before pushing a commit	Skim all three files; tiny edits now save head scratching later.
Starting a new chat thread	Paste the relevant snippet ("See README for quick start, latest CHANGELOG for new features"). That seeds the AI with context instantly.

/5.6 /Why it matters (for you and the AI)

Benefit to you	Benefit to AI / ChatGPT
Onboarding — friends can run the project in minutes.	Reads quick start to generate accurate npm or docker commands.
Traceability — you know when a bug slipped in.	CHANGELOG helps the model suggest rollbacks to a known good version.
Vision tracking — ENHANCEMENTS shows long term goals.	AI suggestions align with roadmap instead of random nice to haves.
Semantic Versioning clarity	Model picks correct version strings (1.0.1 vs 1.1.0).
Less duplicated Q&A	AI answers "How do I run this?" with the steps it finds in README, so you don't retype them.
Jog your memory	Let's be honest, you can't remember every little detail after you come back from a holiday. It's good for you to remind yourself of the amazing work you did!

/5.7. /Best practice cheat sheet

- Write in plain English. Future you may forget today's clever acronym.
- Small, frequent updates. 30 second diary entries beat 30 minute archaeology sessions.
- Follow Keep /a /Changelog headings (Added / Changed / Fixed / Removed). Consistency matters.
- Tree view once per major restructure. If the folder layout changes, regenerate the block.
- Tick or archive completed enhancements. Keeps the wish list motivational, not depressing.
- Reference diary files in prompts. Start a chat with "Per the README quick start..." for laser focused help.
- Back up the project directory. Your diary lives in Git, but the /projects folder should sync to iCloud/OneDrive just in case. Or, ask AI to build you a backup script! (hint: there's one in the resources section)

What /is /a context window, anyway?

Every language model has a “short term memory” bucket called a context window. Picture a rolling scroll of the last N tokens (roughly words + punctuation) the model can see. When the bucket overflows, the oldest stuff falls out—or gets summarised. Bigger windows (Claude /Sonnet’s Ø=Ü¼ /200 k tokens) hold entire PRDs; smaller ones forget after a few pages. /

How size impacts speed & quality

Size	Feels like	Trade off
Small ("d /32 /k)	Snappy responses, cheap	Needs surgical prompts; loses long conversations quickly
Medium (64–128 /k)	Good balance for most coding	Slightly higher latency, higher cost
Huge ("e /200 /k, Claude/Gemini/Codex)	Swallows your 100 page PRD whole	Slower first token, pricey, may over summarise details

I use a M4 Mac /mini with 16 /GB RAM as my dev machine but that isn’t the bottleneck, these limits live on the cloud model’s side. So if replies get vague or AI suggests a “start new chat” banner, it’s usually wanting to prevent the context hitting the ceiling, not your hardware.

Finding your rhythm: one chat per revision

Revision /= code you actually shipped (accepted & maybe committed).

That’s a natural breakpoint to spin up a fresh chat because:

1. The codebase just changed so new files and functions deserve a clean context.
2. You’ll paste the updated README, CHANGELOG, and and what you want from this next stage.

Pro tip: When Cursor nags you mid brainstorm (“Start a new chat?”) but you haven’t accepted any changes yet ignore it. Finish the idea first, then move.

Context windows now are much larger. 200k to 1 million tokens, but there's still something called "Context Rot" where things get worse over time. Claude Code runs a "compacting conversation" process and after each one, things may get worse. So, it's still good practice to update your docs, and then start a new session regardless to keep that context clean.

Best practice cheat sheet

- One chat per revision. New feature? New chat.
- Seed new chats with README, latest CHANGELOG entry, and final two messages from the old chat.

- Preview on localhost before Accept. Orange diff "" committed.
- Reject loudly. Tell the AI why—it learns.
- Restore checkpoint sparingly. Great reset button, but DB/state changes are manual.
- Automate backup. Sync ~/projects to iCloud/OneDrive nightly; checkpoints don't help if the disk dies.

Why write a PRD?

A PRD separates the boys from the men. If you want to build a genuine piece of enterprise-grade, revenue-generating software, you're going to want to take this job seriously and you're going to need a plan. The PRD is your plan to do this right, and sets the foundation. Sure, that simple calculator app didn't need anything like this, but if you're going to build that next SaaS app that generates a few thousand pounds a month for you, there needs to be structure.

A well written PRD is the single cheapest insurance policy you can buy for a software project. Without one, every project becomes a car crash: requirements mutate as they bounce from idea to idea ("oh and we should have that too!") and the finished product often solves a different problem than the one that inspired it. A PRD freezes the conversation in time. It spells out the pain point, the audience, the success metrics, and the things you won't do. Because those decisions are captured in plain English rather than buried in Slack threads or half remembered calls, new teammates (or future you) can absorb months of context in minutes instead of days.

That clarity pays technical dividends too. AI reads the PRD just like a human. When the document says "Front end is React 18, back end is FastAPI, billing runs through Stripe Pro plan," the model stops guessing and starts generating code that actually fits your stack and roadmap. You reduce hallucinated features, trim back and forth prompt cycles, and avoid costly rewrites down the line. In short, a PRD turns abstract vision into a concrete contract between product, engineering, and AI—keeping everyone rowing in the same direction from first commit to final deploy.

Pain without a PRD	Pay off with a PRD
Endless "scope creep" debates	Clear in scope vs out of scope bullets
Five teammates read one Jira ticket five different ways	One doc everyone (and Cursor) can quote
Success criteria invented during the demo	Metrics baked in from day 1
New hire asks "What does this repo even do?"	README shows how to run; PRD shows why it exists

"scope-creep" - when new things get added all the time as they might sound small at the beginning but just get piled on.

8.2 / Core anatomy (take home template - use this to get you started!)

Below is a story driven PRD skeleton you can paste into docs/PRD.md.

```
# Project X – Product /Requirement /Document
_Last updated: 2025 05 03 – v0.4.0_

## /1 /Vision
*Why does this thing deserve to exist?*
Paint a one sentence picture of the world **after** your product succeeds.

> **Example**: “Empower commuters to cut their carbon footprint by visualising
emissions in ...

## /2 /Problem Statement
*What pain are we relieving, and for whom?*
Describe the current frustration or inefficiency in a couple of lines.
Good statements name the **actor**, the **pain**, and the **impact of doing
nothing**.

## /3 / Goals & Success /Metrics
*How will we know we’re winning?*
List 2 4 measurable Key Performance Indicators (KPIs). Tie each to a goal.
```

Goal	KPI	Target
Increase daily engagement	Daily active users	"e /1 /000 by Q3
Validate revenue model	Monthly recurring revenue	"e /\$5 /k by Q4

```
## /4 / Personas & User /Stories
*Who are we building for, and what do they want?*
Start with a sentence long persona profile, then follow with **As /a // /I /want // /So /that
use...

> **Persona – Commuter /Jane**: 29, bikes to work but still drives on weekends.
> **Story US 1**: *As a* commuter, *I want* to log trips, *so that* I can see daily
CO , output.

## /5 /Scope
*Draw the fence so the project doesn’t balloon.*

### / 5.1 /MVP (v1.0)
Features **marked `[MVP]`** must ship in the first public release.

### / 5.2 /Phase /2 (v1.1)
Important, but can wait until the MVP is live.

### / 5.3 /Phase /3 (v2.0+)
Aspirational ideas; keep them visible but out of the critical path.

## /6 / Product /Features & Solutions
*A high level catalogue of how we solve each problem.*
Explain the essence before showing tables.
```

#	Feature	Problem Solved	Phase
F 1	Real time footprint dashboard	Users need instant feedback	[MVP]
F 2	Social sharing widgets	Motivation via peer pressure	2

```

## /7 /Architecture & Tech Stack
*What tools power the solution, and why are they a good fit?*
A paragraph here helps non technical readers follow the tech choices.

> We chose **Next 14 + TypeScript** for SSR performance and developer ergonomics,
**FastAPI ...

Key components at a glance:

- **Front /end:** Next 14, Tailwind /4
- **Back /end:** FastAPI 0.111, Python 3.12
- **DB:** Postgres 16 (managed by Supabase)
- **Auth:** Clerk.dev JWT
- **Infra diagram:** `docs/arch-diagram.png`

## /8 / High Level /Code /Contracts
*Give devs (and AI) the critical interfaces up front.*
Great for strongly typed stacks; keeps everyone (and Cursor) using the same shapes.

// src/api/types/Carbon.ts
export interface CarbonSnapshot {
  kgCO2e: number
  createdAt: Date
}

## /9 / Third Party /Integrations / APIs
*A quick ledger of external systems we depend on, so nobody is surprised by hidden
keys or r...

```

Vendor	Why we use it	Auth mechanism
Stripe	Billing	STRIPE_API_KEY env var
Plaid	Commute data sync	Supabase secrets

```

## /10 /Pricing & Monetisation
*How does this venture pay for lunch?*
Explain tiers, limits, and upgrade paths in prose, then summarise in bullets or a
table.

* **Free:** 100 events // /month, community support
* **Pro ($10 // /mo):** unlimited events, CSV export, team reports
* **Enterprise:** SLA + custom limits (contact sales)

## /11 /Deployment & /Environments
*Where does the code live, and how does it move from laptop to internet?*
Outline CI // /CD flow and rollback strategy before the table.

```

Stage	Host	Branch	URL
Dev	Localhost	any	http://localhost:3000
Test	Vercel Preview	develop	https://carbon dash dev.ver cel.app
Prod	Vercel Prod	main	https://carbon dash.com

```

## /12 /Non Functional /Requirements
*Quality gates that live outside feature checklists.*
These cover speed, security, compliance and accessibility—things users **feel**
even if they...

* **Performance:** 95 % of API calls "d /200 /ms
* **Accessibility:** WCAG 2.2 /AA or better
* **Compliance:** GDPR & SOC /2 Type /II ready
* **Uptime:** 99.9 % monthly SLA

## /13 / Acceptance /Criteria
*Binary conditions that prove a feature is "done".*
Write each as **Given / When / Then** so QA (and AI) can automate tests.

> **AC US 3.1**
> *Given* a logged in user with trips logged,
> *When* they click "Export /CSV",
> *Then* the download starts within 2 /s.

## /14 / Open /Questions & Risks
*A living parking lot for unknowns so nothing slips through the cracks.*

* **Q 1 (Owner /@sam, due /May /15):** Stripe plan IDs final?
* **Risk A:** Plaid API rate limit 100 /req/min — may throttle heavy users.

## /15 / Timeline & /Owners
*Turns intentions into commitments.*
A lightweight table beats a dusty Gantt chart.

```

Phase	Owner	Start	End	Status
MVP build	@dev team	May /6	Jun /20	Ø=Ý on track
Beta launch	@product	Jun /24	Jul /8	#ó pending

```

## /16 / Changelog
*Who changed what in this very PRD.*
Follow **Keep a Changelog** style so diffs stay human readable and AI parsable.

## [0.4.0] — 2025 05 03
### Added
- Pricing & Monetisation section
### Changed
- Tagged dashboard feature as [MVP]

## [0.3.0] — 2025 04 19
### Added
- Architecture diagram reference

```

*Remember: Update the PRD every time scope, metrics or timelines shift.
Cursor reads the latest committed version, so keeping this document current keeps your AI
pair programmer (and teammates) marching in sync with reality.*

Add or drop sections to taste, but keep section numbers stable so future prompts can say “see Section 7”.

/8.3 /Let AI draft the skeleton

Drafting doesn’t have to start with a blinking cursor and writer’s block. Here we show how to hand the outline work to an LLM: one precise prompt produces a neatly formatted scaffold that you can tweak instead of starting from scratch. The AI becomes your junior product manager, and you stay the editor in chief.

1. Start a fresh chat named “PRD draft”.
2. Paste a single directive but obviously fill this out more for what you're after. You can be as lengthy as you want here:

Act as a product manager. Generate a PRD skeleton for a web dashboard that tracks personal carbon emissions. Include sections Vision, Problem, Goals, Personas, MVP Scope, Architecture, Integrations, Pricing, Deployment.

3. Accept the result into docs/PRD.md. Commit: docs: add PRD skeleton v0.1.0.

Now fill each section chapter by chapter, pinging the AI for help:

- User stories: “Add three stories for commuter Jane and one for fleet manager Bob.”
 - Acceptance criteria: “Turn story US 2 into Given/When/Then.”
 - KPIs: “Suggest two measurable success metrics for the Goals section.”
-

/8.4 /Where does the MVP live?

Defining the Minimum /Viable /Product is like drawing a finish line everyone can see. This section explains why the [MVP] tag belongs inside the PRD (rather than a separate doc), how it keeps feature creep at bay, and how AI prompts can filter for those tags to stay laser focused on what must ship first.

MVP = Minimum /Viable /Product — the thinnest slice that proves the idea. Many build an MVP for testing, seeing if people will pay a little to use it, promote it to investors, etc. It's a great first step. "Is this a real thing or not?".

Mark it inside Section /5.1 with [MVP] tags or a dedicated bullet list. Keeping the flag in the PRD:

- Guarantees AI sees what must ship first.
- Lets prompts say “Implement [MVP] items only.”
- Shows stakeholders how Phase /2 features fit the roadmap.

If your team loves punch lists, you can keep a separate MVP Checklist.md, but always reference the PRD for context.

8.5 / Using the PRD during day to day coding

A PRD gains value every time it’s opened during real work. This part demonstrates practical prompt snippets—coding, design, database—that explicitly point the AI to PRD sections. The goal is to make consulting the document second nature, turning it from a static artifact into an always on co pilot.

Task	Prompt snippet
Build a feature	Implement F 1 [MVP] from PRD. Write one integration test. Follow Section /7 stack constraints.
Design pricing page	Using plans in Section /10, scaffold a billing page with Stripe Checkout.
Pick DB schema	Refer to Section /8 code contracts. Create migration for CarbonSnapshot table.

Because the file sits in the repo root (or docs/), AI and you know where it is and you can always say to it, go and read that document.

8.6 / Keeping the PRD up to date

Requirements change, and stale docs are worse than none at all. Here we outline a lightweight governance model—semver bumps, PR reviews, CI checks—that treats the PRD with the same rigor as source files so it never drifts behind reality.

Change	Action
Minor typo/date	Edit, commit chore(prd): fix typo v0.4.1.
New idea (post MVP)	Add bullet in Phase /2; keep [MVP] untouched.
Scope change affects MVP	Bump header to v0.5.0, update CHANGELOG, tag PR feat: expand MVP.
Full pivot	Copy to PRD-v1.0-draft.md, edit there, PR review.

Treat the PRD like code: branches, PRs, CI checks (e.g., fail build if [MVP] tag disappears).

8.7 /Pro tips for a “forever useful” PRD

Once the basics are in place, small habits make the PRD an enduring asset. From embedding diagrams without bloating the repo to listing “Won’t Do” items that repel scope creep, these tips keep the document crisp, actionable, and easy on both humans and AI.

Tip	Pay off
Write for strangers.	New hires onboard via one doc.
Use tables for constants (plan IDs, env vars).	AI copies exact strings, fewer typos.

Embed small diagrams, link big ones.	Context without repo bloat.
List “Won’t do”.	Scope creep shield.
Quarterly reviews.	Direction drifts; doc stays fresh.
Glossary page.	AI (and humans) use terms consistently.
Automate linting.	CI warns if section numbers jump or [MVP] tag misplaced.
Encrypt secrets elsewhere.	.env.example shows names, not values.
Rollback criteria.	AI written deploy scripts know when to abort.

/8.8 / Workflow recap (for you and the AI)

1. Draft skeleton with AI.
2. Fill it out iteratively, asking the AI for each section.
3. Store docs/PRD.md in repo; link it from README.
4. Tag MVP items within the doc.
5. Reference exact sections in prompts (“See Section /9 integrations”).
6. Version & review the PRD like any source file.

Follow that cadence and your PRD becomes a single, always current source of truth: guiding humans, steering AI, and sparing everyone hours of “wait, what are we building again?” discussions.

The “AI Race” — a 30 minute speed run that ties the whole module together. (this is optional of course, but might be a fun thing to try)

Ready to put every trick from Module /2 into practice?

Set a timer for 30 /minutes and race through the checklist below. You’ll start with an empty folder and finish with a tiny, but real web app, complete project diary, and a freshly minted PRD /stub. The goal isn’t perfection; it’s muscle memory.

9.1 /Choose your mini brief (pick one)

Option	One liner	Why it’s good practice
Habit Timer	Countdown widget that reminds you to stand every 25 /min.	Tiny UI + simple JS logic
Weather Emoji	Page that shows current weather as a big emoji using open meteo API.	Requires one API fetch
Tip Tracker	Form that logs restaurant tips and totals daily income.	Introduces light data storage

Write the choice at the top of a sticky note—everything else flows from it.

9.2 / Race checklist (use the section numbers as cues)

#	What to do	Target time
1 Tour	Open your LLM !’ new chat “Race 1”	2 /min
2 /Lab	!’ quick prompt: “Scaffold project / Start local server (npm serve . --port 3005) and open http://localhost:3005 to verify Hello!”	3 /min
3 /Rules	Add cursor rules with two bullets: language + comment style.	2 /min
4 /Workflow	Ask AI to generate UI + basic logic. Preview diff, test live, accept. Commit.	8 /min
5 /Diary	Create README (with tree view), CHANGELOG (0.0.1 – initial UI),	5 /min
6 /Extra /Brains	Now ask for UX improvements. ENHANCEMENTS (dark mode). and implement.	4 /min
7 /Optimise	New chat “Race 2”; seed with README + last fix. Add one refinement (e.g., form validation).	3 /min
8 /PRD stub	Prompt AI: “Draft PRD skeleton for this Habit Timer; tag MVP items.”	3 /min

Stop timer — 30 /minutes max!

9.3 / Self scoring rubric (bonus gamification)

Points	Milestone
+2	App loads in browser with no console errors.
+2	.rules committed.

+2	README has folder tree.
+3	CHANGELOG shows version /0.0.1.
+3	PRD skeleton exists with Vision, Scope, MVP tags.
+3	At least one AI suggestion accepted.
+5	All commits pushed to a GitHub repo named app race.
" 1 / bug	Deduct for each unhandled error in console or broken link.

Perfect score: 20. Brag about it in your next TikTok!

9.4 /Debrief prompts (reflect & improve)

1. What slowed you down the most, prompt clarity, file paths, or testing?
2. Did the .cursorrules actually influence AI output?
3. Where did context window nag pop up? How did you handle it?

Paste these Qs into a new Ask mode chat and let Cursor critique your race. The feedback becomes a learning loop for the next module.

Why this matters

The “AI race” compresses everything you learned:

- You spin up a local lab in seconds.
- Rules steer the AI immediately.
- Diff workflow lets you test before you trust.
- A living project diary keeps context tight.
- Optimisation hacks (new chat per revision) avoid token bloat.
- Even a rough cut PRD ensures tomorrow’s work aligns with today’s goal.

Hit start on the timer and watch how much you can build confidently before the alarm rings. That speed and structure is exactly what the rest of the curriculum will build on.